

AI ASSISTED CODING

T.Snuhith Reddy

2303A510H9

BATCH – 03

24 – 03 – 2026

ASSINGMENT – 17.2

LAB – 17 : AI for Data Processing : Data Cleaning and Preprocessing Scripts.

Task – 01 : Student Academic Data Cleaning.

Prompt : Generate a Python script using Pandas to clean student academic data by handling missing values, removing duplicates, standardizing text fields, and encoding categorical variables.

Code :

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder

data = pd.read_csv("student_data.csv")

num_cols = data.select_dtypes(include=['int64', 'float64']).columns
data[num_cols] = data[num_cols].fillna(data[num_cols].mean())

cat_cols = data.select_dtypes(include=['object']).columns
for col in cat_cols:
    data[col].fillna(data[col].mode()[0], inplace=True)
data.drop_duplicates(inplace=True)
for col in cat_cols:
    data[col] = data[col].str.lower().str.strip()
le = LabelEncoder()
for col in cat_cols:
    data[col] = le.fit_transform(data[col])

data['marks'] = (data['G1'] + data['G2'] + data['G3']) / 3
data['attendance'] = 100 - data['absences']
print("\nCleaned Data:\n", data.head())
```

Output :

```
...
Cleaned Data:
   school  sex  age  address  famsize  Pstatus  Medu  Fedu  Mjob  Fjob  ... \
0        0    0   18        1        0        0      4      4      0      4  ...
1        0    0   17        1        0        1      1      1      0      2  ...
2        0    0   15        1        1        1      1      1      0      2  ...
3        0    0   15        1        0        1      4      2      1      3  ...
4        0    0   16        1        0        1      3      3      2      2  ...

   goout  Dalc  Walc  health  absences  G1  G2  G3  marks  attendance
0      4      1      1      3         6   5   6   6   5.666667         94
1      3      1      1      3         4   5   5   6   5.333333         96
2      2      2      3      3        10   7   8  10   8.333333         90
3      2      1      1      5         2  15  14  15  14.666667         98
4      2      1      2      5         4   6  10  10   8.666667         96
```

Data Set Used : Student_data[Kaggle].

Explanation : This dataset contains multiple academic and personal features, so missing values are handled using mean and mode. Duplicate rows are removed and text fields are standardized. All categorical columns are encoded using Label encoder. New features like marks are created. The final dataset is clean and ready for analysis or ML models.

Task – 02 : Financial Transaction Data Preprocessing.

Prompt : Generate Python code to preprocess financial transaction data by converting dates, extracting time features, normalizing amounts, and handling outliers.

Code :

```
TASK 02

[14] import pandas as pd
from sklearn.preprocessing import MinMaxScaler

data = pd.read_csv("sample_dataset.csv")
data.rename(columns={'Date': 'transaction_date', 'Transaction Amount': 'amount'}, inplace=True)
data['transaction_date'] = pd.to_datetime(data['transaction_date'], errors='coerce')
data['month'] = data['transaction_date'].dt.month
data['year'] = data['transaction_date'].dt.year
data.ffill(inplace=True)

scaler = MinMaxScaler()
data[['amount']] = scaler.fit_transform(data[['amount']])
q1 = data['amount'].quantile(0.25)
q3 = data['amount'].quantile(0.75)
iqr = q3 - q1

lower = q1 - 1.5 * iqr
upper = q3 + 1.5 * iqr
data = data[(data['amount'] >= lower) & (data['amount'] <= upper)]

print("\nProcessed Data:\n", data.head())
```

Output :

Processed Data:							
...	Customer ID	Name	Surname	Gender	Birthdate	amount	\
0	752858	Sean	Rodriguez	F	2002-10-20	0.010171	
2	305449	Jacob	Williams	M	1981-10-25	0.037050	
3	988259	Nathan	Snyder	M	1977-10-26	0.002104	
4	764762	Crystal	Knapp	F	1951-11-02	0.019099	
5	576539	Monica	Bartlett	F	2001-10-20	0.031430	
	transaction_date	Merchant Name		Category	month	year	
0	2023-04-03	Smith-Russell		Cosmetic	4	2023	
2	2023-09-20	Steele Inc		Clothing	9	2023	
3	2023-01-11	Wilson, Wilson and Russell		Cosmetic	1	2023	
4	2023-06-13	Palmer-Hinton		Electronics	6	2023	
5	2023-08-24	Tran, Torres and Joyce		Cosmetic	8	2023	

Data Set Used : Sample_data[Kaggle].

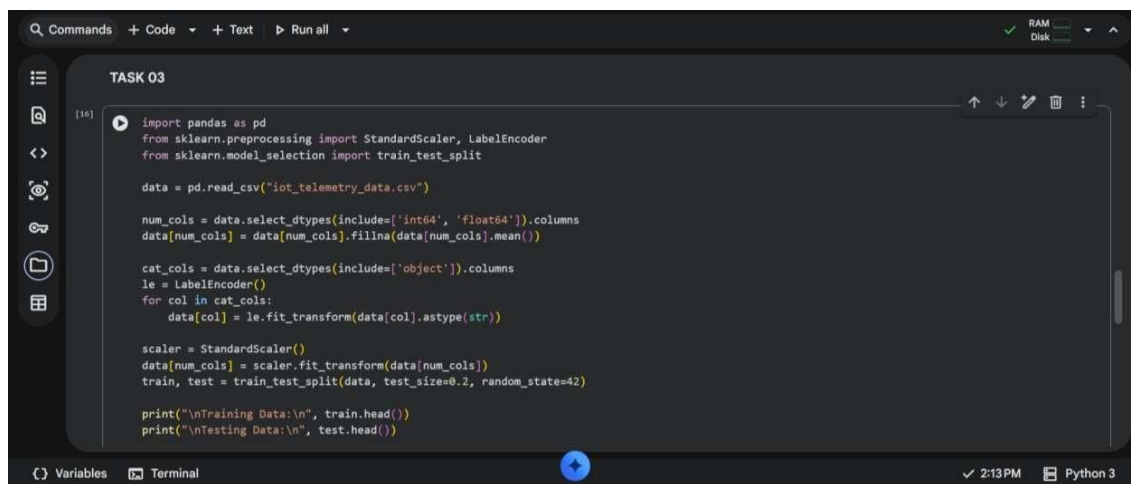
Explanation : The dataset is first loaded and the transaction date column is converted into datetime format for easier time-based analysis. New features like month and year are extracted from the date to enhance insights. Missing values are handled

using forward fill to maintain data continuity. The transaction amount is normalized using MinMaxScaler to bring values into a common range. Finally, outliers are removed using the IQR method to improve data quality and model performance.

Task – 03 : Environmental Sensor Data Preparation.

Prompt : Write Python code to clean sensor data, handle missing values, scale numerical features, encode categorical variables, and split the dataset.

Code :

A screenshot of a Jupyter Notebook interface. The top bar shows 'Commands', '+ Code', '+ Text', and 'Run all'. The left sidebar has icons for file explorer, search, and other notebook functions. The main area is titled 'TASK 03' and contains the following Python code:

```
[16]: import pandas as pd
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.model_selection import train_test_split

data = pd.read_csv("iot_telemetry_data.csv")

num_cols = data.select_dtypes(include=['int64', 'float64']).columns
data[num_cols] = data[num_cols].fillna(data[num_cols].mean())

cat_cols = data.select_dtypes(include=['object']).columns
le = LabelEncoder()
for col in cat_cols:
    data[col] = le.fit_transform(data[col].astype(str))

scaler = StandardScaler()
data[num_cols] = scaler.fit_transform(data[num_cols])
train, test = train_test_split(data, test_size=0.2, random_state=42)

print("\nTraining Data:\n", train.head())
print("\nTesting Data:\n", test.head())
```

The bottom of the notebook shows 'Variables' and 'Terminal' tabs, with a status bar indicating '2:13 PM' and 'Python 3'.

Data Set Used : Iot_Telemetry_Data[Kaggle].

Explanation :

The dataset is loaded and missing numerical values are filled using the mean to ensure completeness. Categorical columns are encoded into numerical form using LabelEncoder. All numerical features are standardized using StandardScaler to improve model performance. The dataset is then split into training and testing sets for machine learning tasks. This preprocessing makes the data clean, consistent, and ready for predictive modeling.

Output :

```

Training Data:
... 403743  1.718761    2  1.049533 -0.977585  False  1.036094  False
89477   -0.965176    0  0.191110  1.239461  False  0.225863  False
285682  0.709553    1 -0.503622  0.808369   True -0.466690  False
45954   -1.339768    2  0.531880 -0.775235  False  0.552874  False
389117  1.593869    0 -1.937926  1.371428  False -2.051467  False

      smoke      temp
403743  1.039346 -0.353546
89477   0.219849 -1.242980
285682 -0.473833  1.277084
45954   0.549630 -0.131187
389117 -2.032107 -1.168861

Testing Data:
362352  1.370019    1 -0.237835 -0.440919   True -0.197264  False
382697  1.540262    2  0.843570 -0.766438  False  0.845664  False
386329  1.570416    1 -0.345070  0.438861   True -0.305255  False
141680 -0.517428    0 -1.438976  1.652958  False -1.469927  False
212318  0.081979    2  0.595091 -1.162339  False  0.612720  False

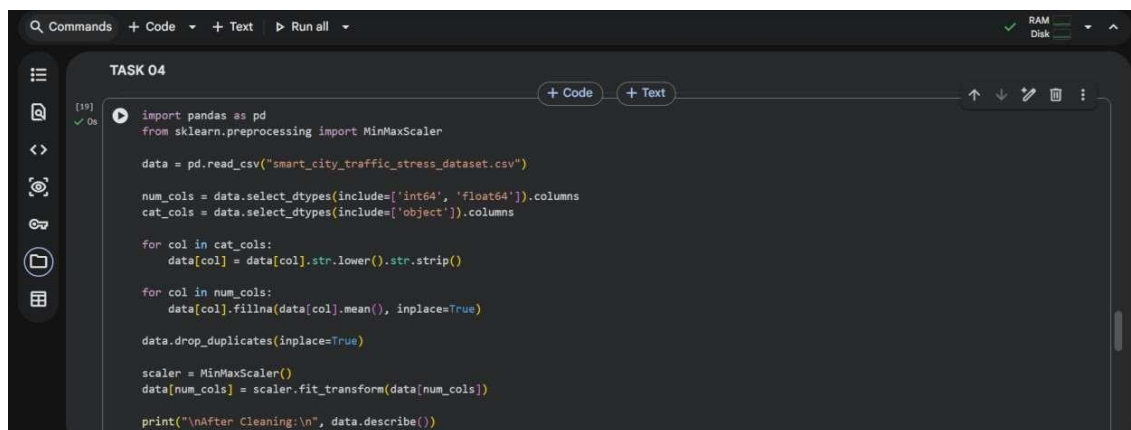
      smoke      temp
362352 -0.204768  2.351818
382697  0.846025 -0.242366

```

Task – 04 : Real Time Application : Smart City Traffic Data Cleaning.

Prompt : Generate Python code to clean traffic data by standardizing text, filling missing values, removing duplicates, normalizing features, and comparing dataset quality.

Code :



```

TASK 04

import pandas as pd
from sklearn.preprocessing import MinMaxScaler

data = pd.read_csv("smart_city_traffic_stress_dataset.csv")

num_cols = data.select_dtypes(include=['int64', 'float64']).columns
cat_cols = data.select_dtypes(include=['object']).columns

for col in cat_cols:
    data[col] = data[col].str.lower().str.strip()

for col in num_cols:
    data[col].fillna(data[col].mean(), inplace=True)

data.drop_duplicates(inplace=True)

scaler = MinMaxScaler()
data[num_cols] = scaler.fit_transform(data[num_cols])

print("\nAfter Cleaning:\n", data.describe())

```

Data Set Used : Smart_city_traffic_stress_dataset[Kaggle].

Output :

After Cleaning:					
	traffic_density	horn_events_per_min	avg_speed	signal_wait_time	\
count	50000.000000	50000.000000	50000.000000	50000.000000	
mean	0.499267	0.354817	0.530562	0.463345	
std	0.291048	0.176126	0.179240	0.232271	
min	0.000000	0.000000	0.000000	0.000000	
25%	0.247706	0.222848	0.386131	0.264173	
50%	0.495413	0.350563	0.530470	0.464173	
75%	0.752294	0.479083	0.675335	0.660144	
max	1.000000	1.000000	1.000000	1.000000	

	road_quality_score	stress_index
count	50000.000000	50000.000000
mean	0.660046	0.444023
std	0.209512	0.163516
min	0.000000	0.000000
25%	0.516667	0.316200
50%	0.666667	0.436700
75%	0.816667	0.562700
max	1.000000	1.000000

Explanation :

The dataset is cleaned by standardizing text fields to ensure consistency across categorical data. Missing numerical values are filled using the mean to avoid data loss. Duplicate records are removed to improve data accuracy. Numerical features are normalized using MinMaxScaler for better comparability. The dataset summary before and after cleaning highlights improvements in data quality.

Task – 05 : Social Media Text Data Cleaning and Feature Extraction.

Prompt : Write Python code to clean social media text data by removing duplicates, cleaning text, tokenizing, removing stopwords, and extracting features.

Code :

```

TASK 05
[22] import pandas as pd
import re
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer
import nltk

data = pd.read_csv("mental_health_social_media_dataset.csv")

data.drop_duplicates(inplace=True)
text_col = data.select_dtypes(include=['object']).columns[0]
data.dropna(subset=[text_col], inplace=True)

def clean_text(text):
    text = re.sub(r"http\S+", "", str(text))
    text = re.sub(r"[^a-zA-Z]", " ", text)
    text = text.lower()
    return text

data['clean_text'] = data[text_col].apply(clean_text)
nltk.download('stopwords') # Download the stopwords corpus
stop_words = set(stopwords.words('english'))
ps = PorterStemmer()
data['tokens'] = data['clean_text'].apply(lambda x: [ps.stem(word) for word in x.split() if word not in stop_words])

data['word_count'] = data['tokens'].apply(len)
print("\nProcessed Data:\n", data.head())

```

Output :

Processed Data:									
...	person_name	age	date	gender	platform	daily_screen_time_min \			
0	Reyansh Ghosh	35	1/1/2024	Male	Instagram	320			
1	Neha Patel	24	1/12/2024	Female	Instagram	453			
2	Ananya Naidu	26	1/6/2024	Male	Snapchat	357			
3	Neha Das	66	1/17/2024	Female	Snapchat	190			
4	Reyansh Banerjee	31	1/28/2024	Male	Snapchat	383			
	social_media_time_min	negative_interactions_count		\					
0	160			1					
1	226			1					
2	196			1					
3	105			0					
4	211			1					
	positive_interactions_count	sleep_hours	physical_activity_min \						
0	2	7.4	28						
1	3	6.7	15						
2	2	7.2	24						
3	1	8.0	41						
4	2	7.1	22						
	anxiety_level	stress_level	mood_level	mental_state	clean_text \				
0	2	7	6	Stressed	reyansh ghosh				
1	3	8	5	Stressed	neha patel				
2	3	7	6	Stressed	ananya naidu				
3	2	6	6	Stressed	neha das				
4	3	7	6	Stressed	reyansh banerjee				

Data Set Used : Mental_health_social_media_dataset[Kaggle].

Explanation :

The dataset is cleaned by removing duplicate and missing text entries. Text data is processed by eliminating URLs, special characters, and converting everything to lowercase. Stopwords are removed and stemming is applied to simplify words. The cleaned text is tokenized into meaningful words. Additional features like word count are generated to support sentiment analysis tasks.